



APRENDIZAGEM DE MÁQUINA

Otávio Calaça Xavier



INF

INSTITUTO DE
INFORMÁTICA



UFG

UNIVERSIDADE
FEDERAL DE GOIÁS



PODER JUDICIÁRIO

Tribunal de Justiça do Estado de Goiás



Programa de
Residência em TI

Aula 4

Aprendizado

Não-Supervisionado

INF

INSTITUTO DE
INFORMÁTICA



UFMG
UNIVERSIDADE
FEDERAL DE GOIÁS



PODER JUDICIÁRIO

Tribunal de Justiça do Estado de Goiás



Programa de
Residência em TI

Objetivos da Aula

- Compreender o conceito de **aprendizado não supervisionado**
- Explorar as técnicas de **clusterização** e **redução de dimensionalidade**
- Aplicar **K-Means** e **PCA** a dados reais (Wine e Digits)
- Avaliar a qualidade dos clusters com ***Silhouette Score*** (Silhueta)

O que é Aprendizado Não Supervisionado

- O modelo **aprende sozinho a estrutura dos dados**
- Não há variável de saída (*target*)
- Objetivo: **descobrir padrões ocultos**
- Exemplos:
 - Agrupar dados semelhantes
 - Reduzir dimensionalidade
 - Detectar anomalias e outliers

Aplicações no Tribunal de Justiça

- Agrupar **processos semelhantes** por tipo ou assunto
- Detectar **casos atípicos** (ex.: trâmites muito longos)
- Descobrir **perfis de produtividade** entre varas
- Agrupar **decisões** com fundamentos jurídicos parecidos
- ...

Principais Técnicas

- **Clusterização**

- K-Means
- DBSCAN
- Hierarchical Clustering

- **Redução de Dimensionalidade**

- PCA (Principal Component Analysis)
- t-SNE / UMAP (para visualização)

Clusterização

- Agrupa dados semelhantes em **clusters**
- Cada grupo contém **itens próximos** entre si
- Descubra **padrões ocultos** sem usar rótulos
- Útil para **segmentação, detecção de padrões e outliers**
- Pode usar **distância, densidade** ou **hierarquia**
- Exemplo de métodos: **K-Means, DBSCAN, Hierarchical**

K-Means – Agrupamento por Similaridade

- Agrupa dados em **K clusters** definidos por **centroides**
- Minimiza a distância média entre pontos e o centroide
- Exige definir **K** previamente
- Sensível à **escala** e à **inicialização**
- Rápido e fácil de implementar
- Aplicações: **segmentação, perfis, documentos**

K-Means – Agrupamento por Similaridade

- Exemplo

```
X, _ = make_blobs(n_samples=300, centers=4,
                  random_state=42)
```

```
# Aplicando K-Means
```

```
kmeans = KMeans(n_clusters=4, random_state=42)
```

```
labels = kmeans.fit_predict(X)
```

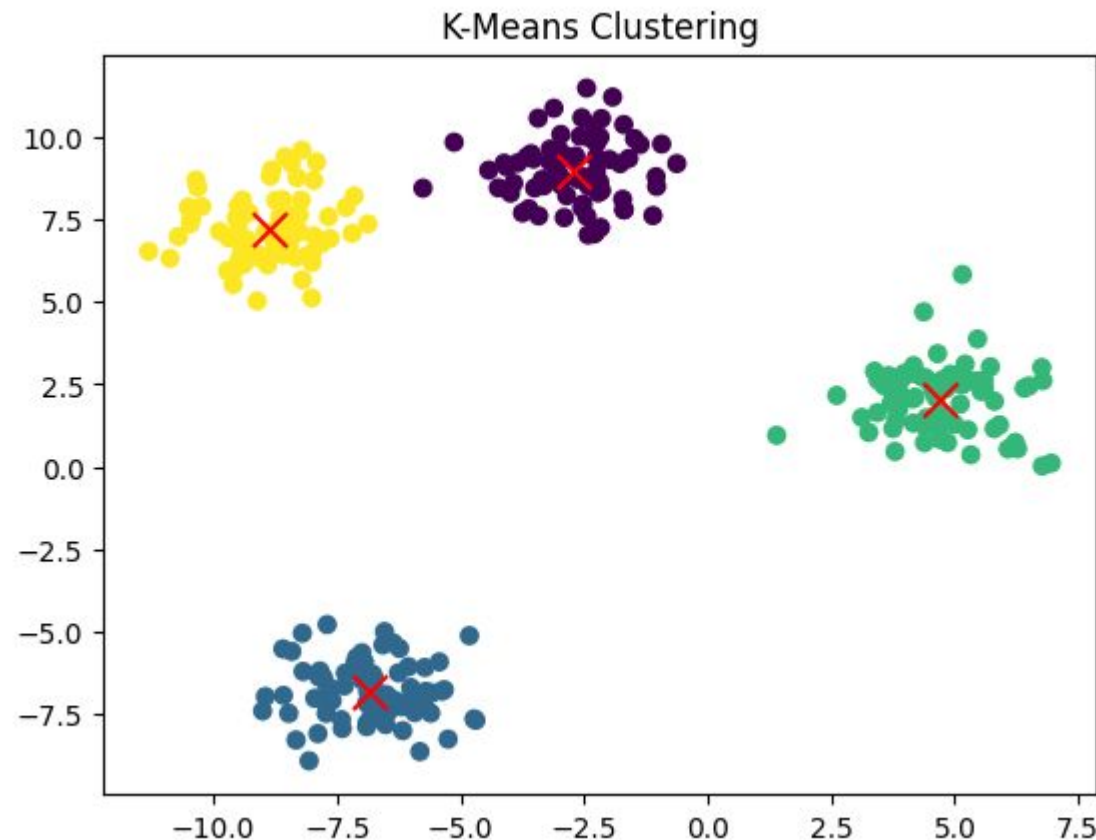
```
# Visualização
```

```
plt.scatter(X[:,0], X[:,1], c=labels, cmap='viridis')
```

```
plt.scatter(kmeans.cluster_centers_[0],
            kmeans.cluster_centers_[1],
            c='red', marker='x', s=150)
```

```
plt.title('K-Means Clustering')
```

```
plt.show()
```



DBSCAN – Agrupamento por Densidade

- Agrupa pontos densamente conectados
- Detecta **outliers** automaticamente
- Não precisa definir número de clusters
- Depende dos parâmetros **eps** e **min_samples**
- Lida bem com **formas não lineares**
- Aplicações: **fraudes, anomalias, ruído**

DBSCAN – Agrupamento por Densidade

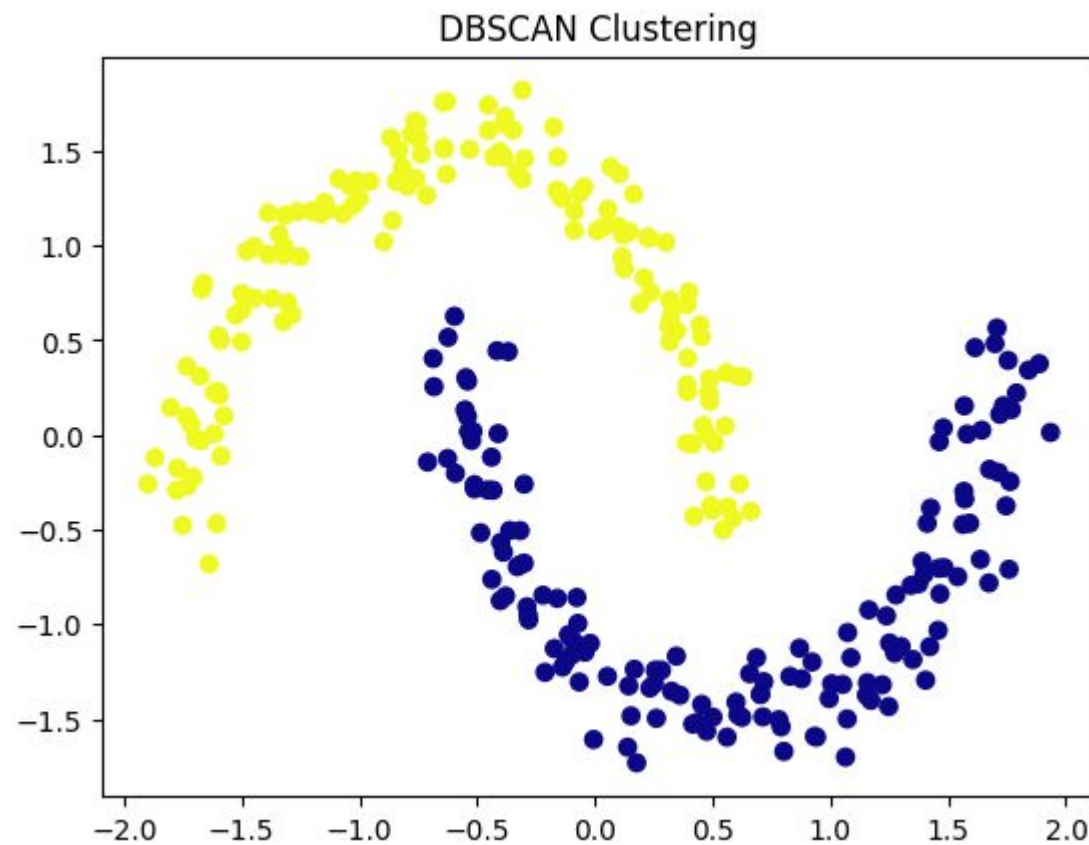
- Exemplo

```
# Dados com formato não linear
X, _ = make_moons(n_samples=300, noise=0.07,
                  random_state=42)

X = StandardScaler().fit_transform(X)

# Aplicando DBSCAN
db = DBSCAN(eps=0.3, min_samples=5)
labels = db.fit_predict(X)

# Visualização
plt.scatter(X[:,0], X[:,1], c=labels, cmap='plasma')
plt.title('DBSCAN Clustering')
plt.show()
```



Clusterização Hierárquica

- Constrói uma **árvore de similaridade (dendrograma)**
- Une ou divide clusters **progressivamente**
- Não requer número de clusters inicial
- Oferece **estrutura interpretável** dos dados
- Custo alto para bases grandes
- Aplicações: **grupos de textos, perfis, temas**

Clusterização Hierárquica

- Exemplo

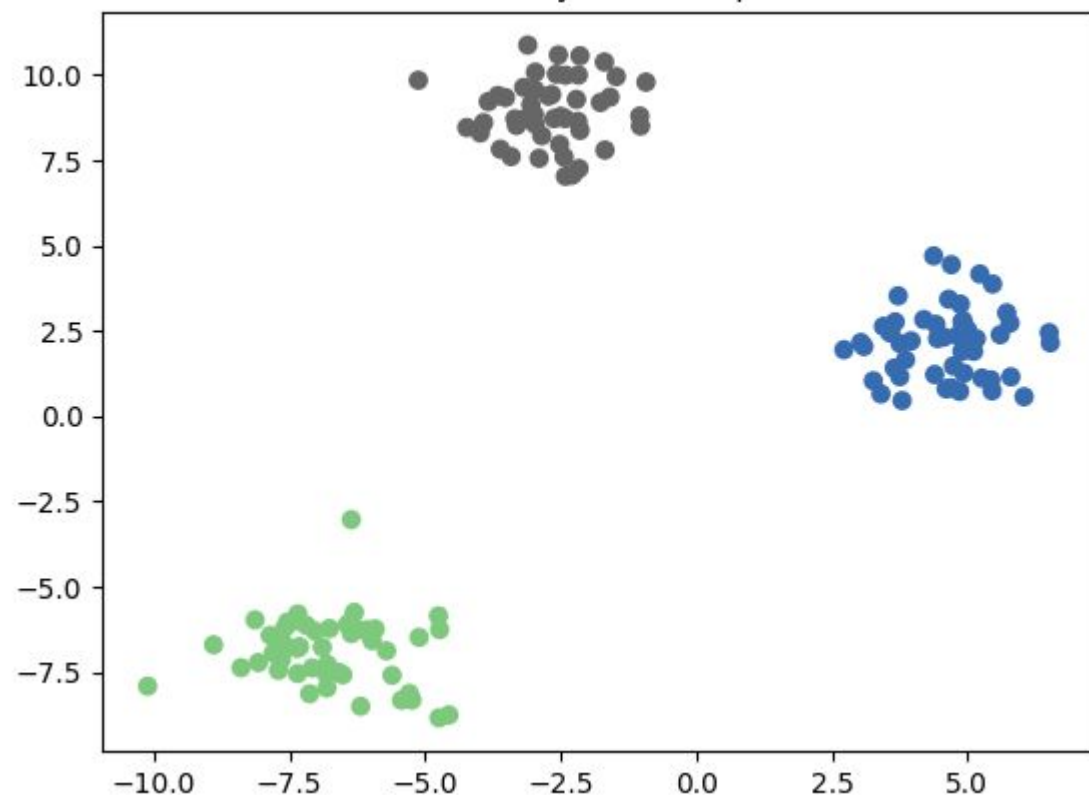
```
# Dados simulados
X, _ = make_blobs(n_samples=150, centers=3,
                  random_state=42)

# Clusterização hierárquica
hc = AgglomerativeClustering(n_clusters=3,
                             linkage='ward')

labels = hc.fit_predict(X)

# Visualização
plt.scatter(X[:,0], X[:,1], c=labels, cmap='Accent')
plt.title('Clusterização Hierárquica')
plt.show()
```

Clusterização Hierárquica



Redução de Dimensionalidade

- Resume dados em **menos variáveis (*features*)**
- Mantém o máximo de **informação relevante**
- Ajuda na **visualização e desempenho** de modelos
- Reduz **ruído e correlação** entre atributos
- Pode ser **linear (PCA)** ou **não linear (t-SNE, UMAP)**
- Facilita análise e **interpretação de padrões**

PCA – Análise de Componentes Principais

- Reduz dimensões mantendo **variância máxima**
- Gera **componentes não correlacionados**
- Melhora eficiência e **visualização** dos dados
- Remove **ruído e redundância**
- Método **linear** mais usado em ML
- Aplicações: **pré-processamento, visualização**

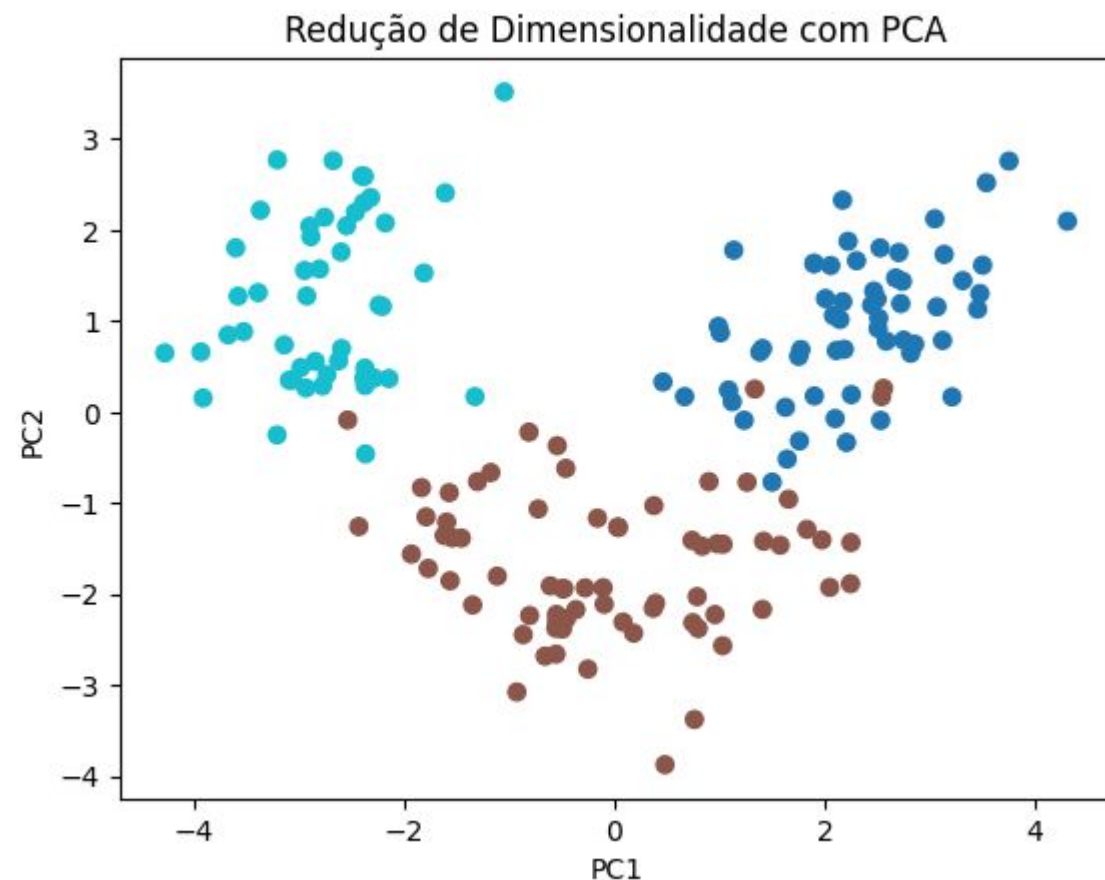
PCA – Análise de Componentes Principais

- Exemplo

```
# Dados do Wine
wine = load_wine()
X = StandardScaler().fit_transform(wine.data)

# Aplicando PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

# Visualização
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=wine.target,
            cmap= 'tab10')
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.title('Redução de Dimensionalidade com PCA')
plt.show()
```



t-SNE e UMAP – Visualização de Alta Dimensão



- Técnicas **não lineares** de redução de dimensões
- Preservam **relações locais** e estrutura dos dados
- **t-SNE**: visualmente detalhado, mas lento
- **UMAP**: rápido e preserva contexto global
- Ideais para **embeddings e análise exploratória**
- Usados em **texto, imagem e dados complexos**



t-SNE e UMAP – Visualização de Alta Dimensão



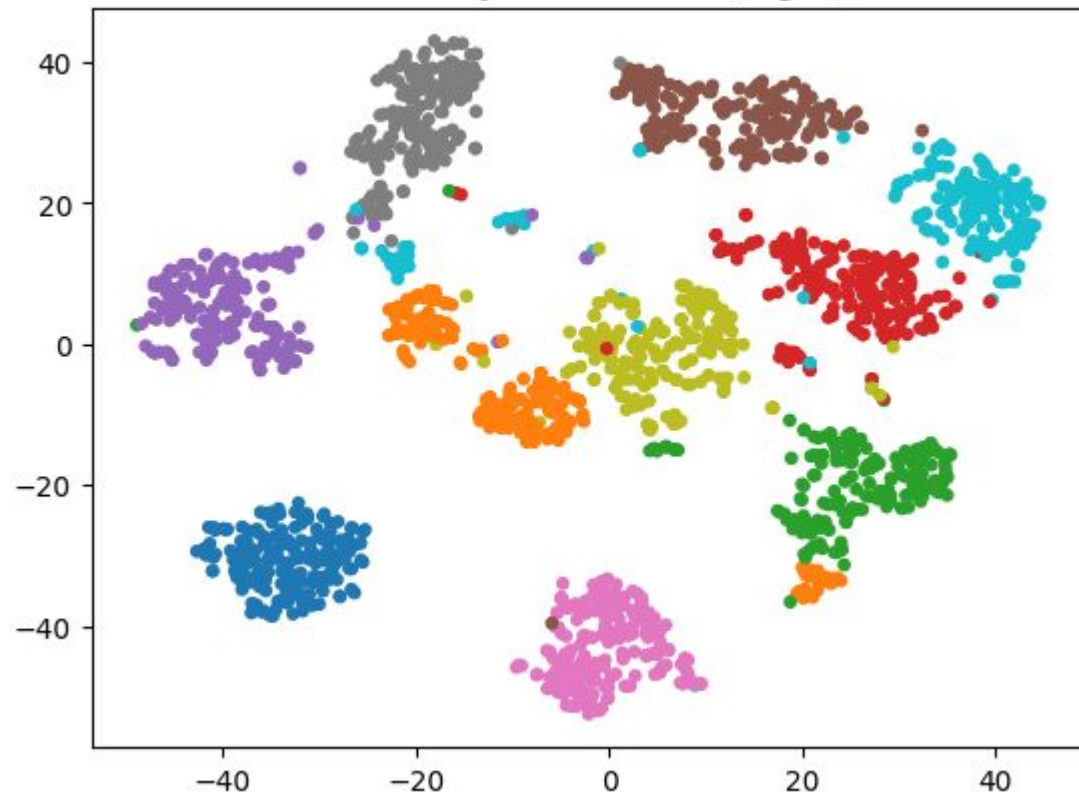
- Exemplo

```
# Dados de dígitos
digits = load_digits()
X = StandardScaler().fit_transform(digits.data)

# Redução com t-SNE
tsne = TSNE(n_components=2, perplexity=30,
            random_state=42)
X_tsne = tsne.fit_transform(X)

# Visualização
plt.scatter(X_tsne[:, 0], X_tsne[:, 1], c=digits.target,
            cmap='tab10', s=15)
plt.title('Visualização com t-SNE (Digits)')
plt.show()
```

Visualização com t-SNE (Digits)



Método do Cotovelo (Elbow Method)

- O Elbow Method ajuda a escolher o número ideal de clusters (**k**) no K-Means.
- A ideia é:
 - a. Rodar o K-Means com diferentes valores de **k**
 - b. Calcular a **inércia** (a soma das distâncias quadradas aos centroides)
 - c. Procurar o ponto onde a redução da inércia começa a desacelerar — o “cotovelo”
-  Esse ponto indica o equilíbrio entre compactação dos clusters e simplicidade do modelo.

Método do Cotovelo (Elbow Method)

- Exemplo

```
# Gerando dados simulados
```

```
X, _ = make_blobs(n_samples=300, centers=5, cluster_std=0.6, random_state=0)
```

```
# Calculando a inércia para diferentes valores de k
```

```
inertia = []
```

```
k_values = range(1, 11)
```

```
for k in k_values:
```

```
    kmeans = KMeans(n_clusters=k, random_state=0)
```

```
    kmeans.fit(X)
```

```
    inertia.append(kmeans.inertia_)
```

```
# Plotando o Elbow Method
```

```
plt.figure(figsize=(8, 5))
```

```
plt.plot(k_values, inertia, '-o', color='#8d0801')
```

```
plt.title('Elbow Method', fontsize=14)
```

```
plt.xlabel('Número de Clusters (k)', fontsize=12)
```

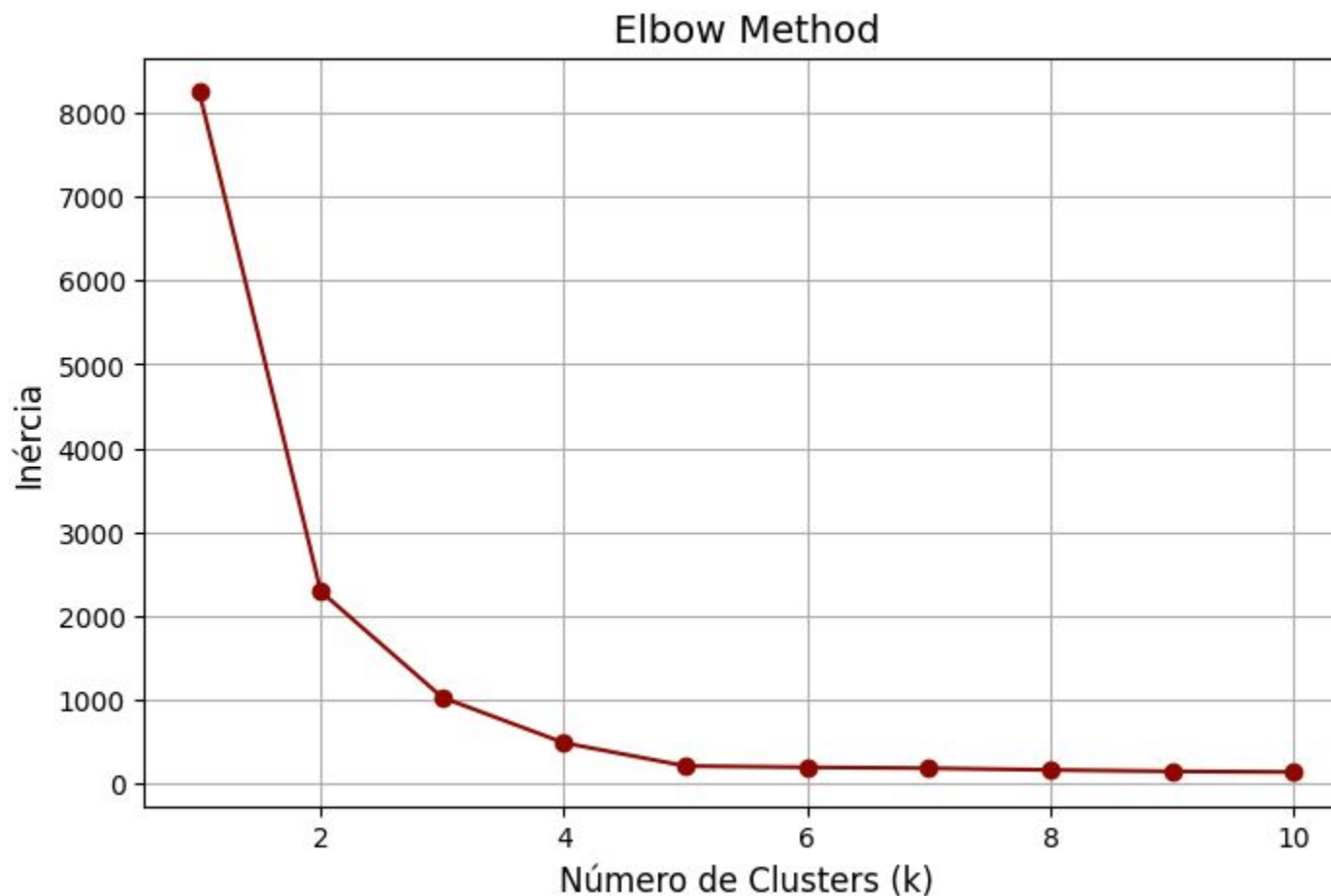
```
plt.ylabel('Inércia', fontsize=12)
```

```
plt.grid()
```

```
plt.show()
```


Método do Cotovelo (Elbow Method)

- Exemplo



Interpretando o Gráfico do Cotovelo

- A inércia **sempre diminui** conforme o número de clusters aumenta.
- Porém, após certo ponto, **o ganho se torna pequeno** → o cotovelo.
- Esse ponto indica o **k ideal** (melhor separação sem complexidade desnecessária).

💡 No exemplo, o “cotovelo” aparece próximo de **k = 5**, que é justamente o número de centros reais usados no ***make_blobs***.

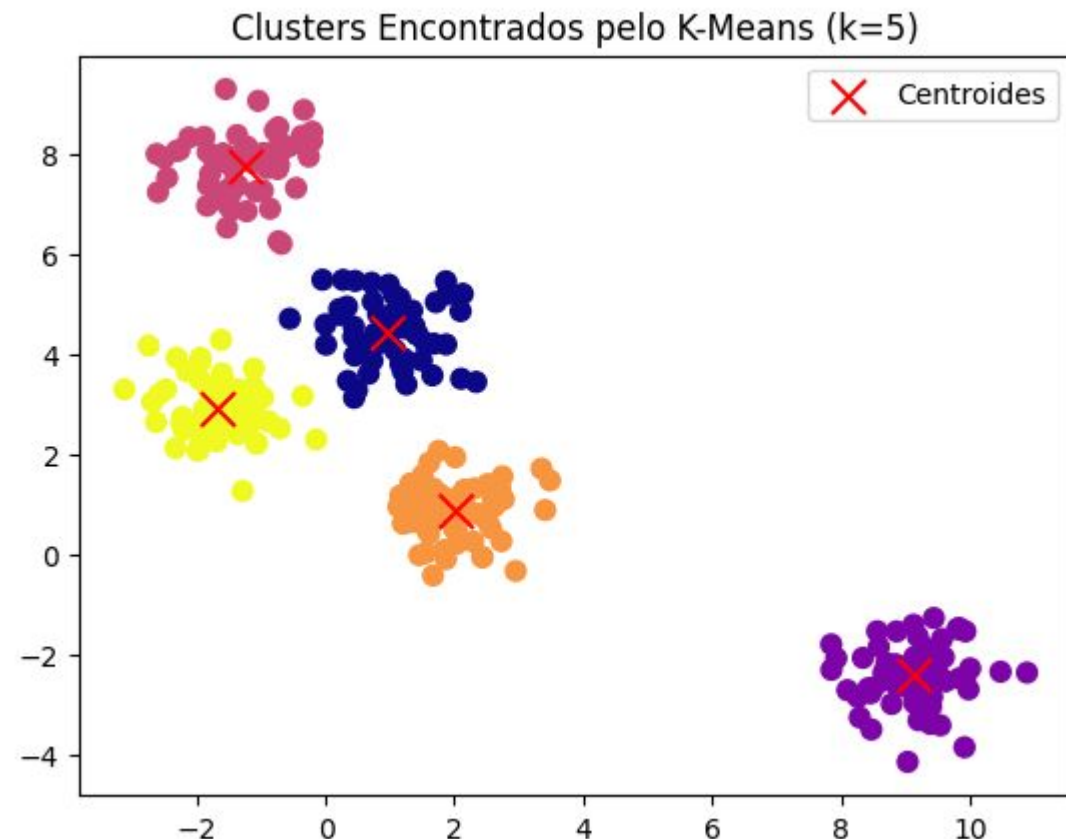
Método do Cotovelo (Elbow Method)

- Exemplo

```
kmeans = KMeans(n_clusters=5, random_state=0)
y_kmeans = kmeans.fit_predict(X)

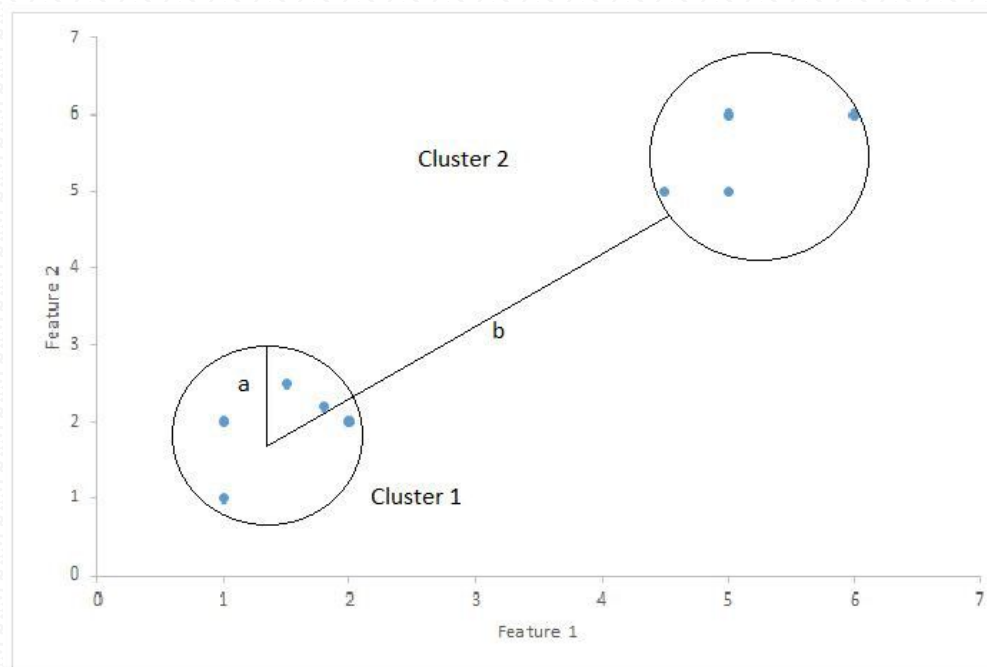
plt.scatter(X[:, 0], X[:, 1], c=y_kmeans, cmap='plasma',
            s=50)
plt.scatter(kmeans.cluster_centers_[0, 0],
            kmeans.cluster_centers_[0, 1],
            c='red', marker='x', s=150,
            label='Centroides')

plt.title('Clusters Encontrados pelo K-Means (k=5)')
plt.legend()
plt.show()
```



Não Supervisionado - Métricas de Avaliação

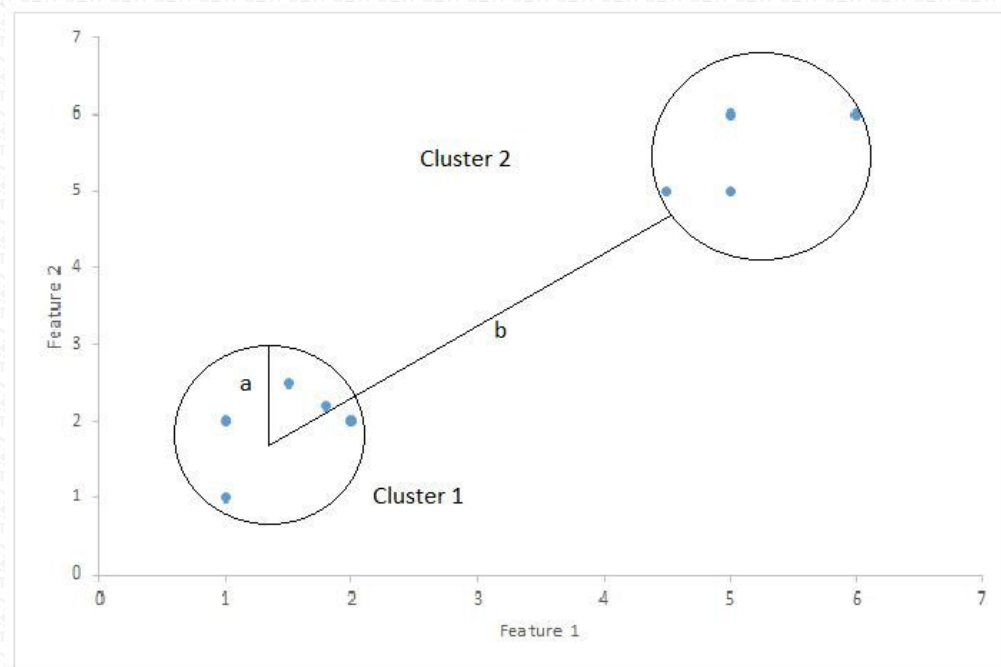
Nesse caso, não há **target** ou **label**. Logo, a avaliação acaba sendo relacionada à “coesão” dos clusters.



Não Supervisionado - Métricas de Avaliação

Silhouette Coefficient

- Usa a distância média intra-cluster (a) e a distância média para com o cluster vizinho mais próximo (b) para cada item.
- $(b - a) / \max(a, b)$.
- Só pode ser definido se:
 $2 \leq n_clusters \leq n_amostras - 1$.



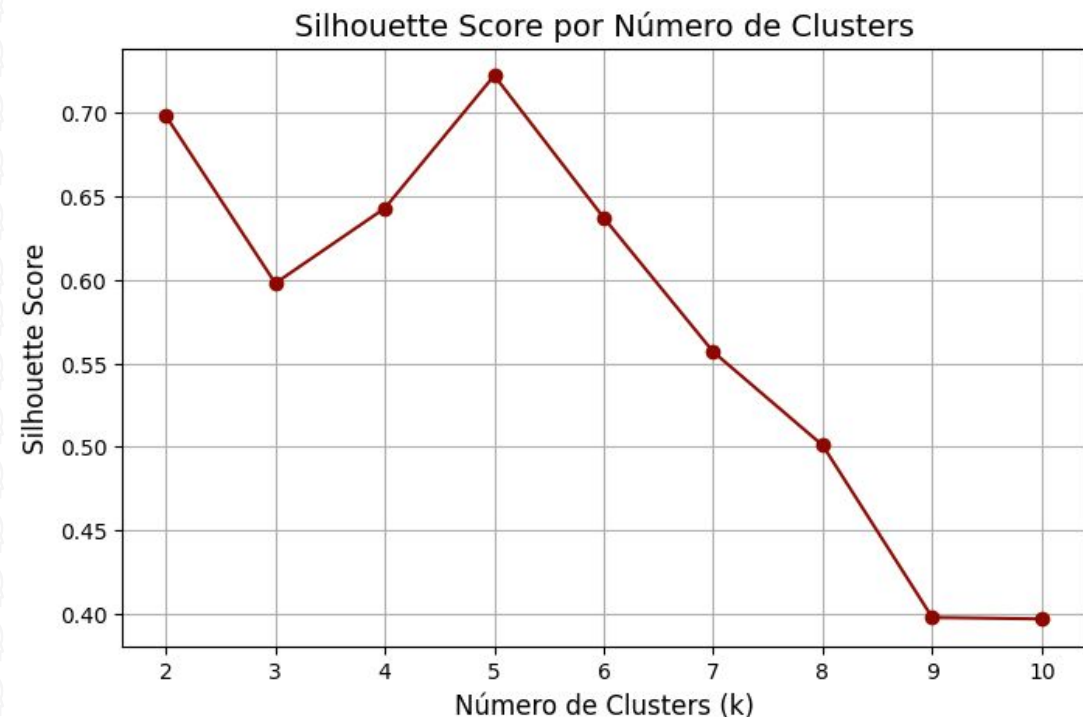
t-SNE e UMAP – Visualização de Alta Dimensão



- Exemplo

```
# Calculando o Silhouette Score para diferentes valores  
# de k
```

```
silhouette_scores = []  
for k in range(2, 11):  
    kmeans = KMeans(n_clusters=k, random_state=0)  
    kmeans.fit(X)  
    score = silhouette_score(X, kmeans.labels_)  
    silhouette_scores.append(score)
```



```
# Plotando o Silhouette Score  
plt.figure(figsize=(8, 5))  
plt.plot(range(2, 11), silhouette_scores, '-o', color='#8d0801')  
plt.title('Silhouette Score por Número de Clusters',  
         fontsize=14)  
plt.xlabel('Número de Clusters (k)', fontsize=12)  
plt.ylabel('Silhouette Score', fontsize=12)  
plt.grid()
```

Atividades

1. Aplique PCA + K-Means ao dataset MNIST (digits com maior resolução)
2. Varie o número de componentes da PCA (2, 10, 30)
3. Avalie usando Silhouette Score e comparando com os *targets* originais
4. Visualize e interprete os clusters